Since 2004
UET
ĐẠI HỌC CÔNG NGHỆ, ĐHQGHN
VNU-University of Engineering and Technology

Since 1906
VNU
ĐẠI HỌC QUỐC GIA HÀ NỘI
Vietnam National University, Hanoi

# INT3405 - Machine Learning

## Lecture 3: Linear Regression

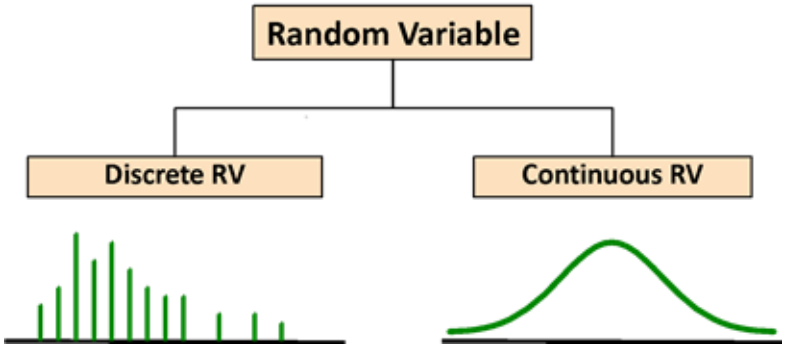
**Viet-Cuong Ta**

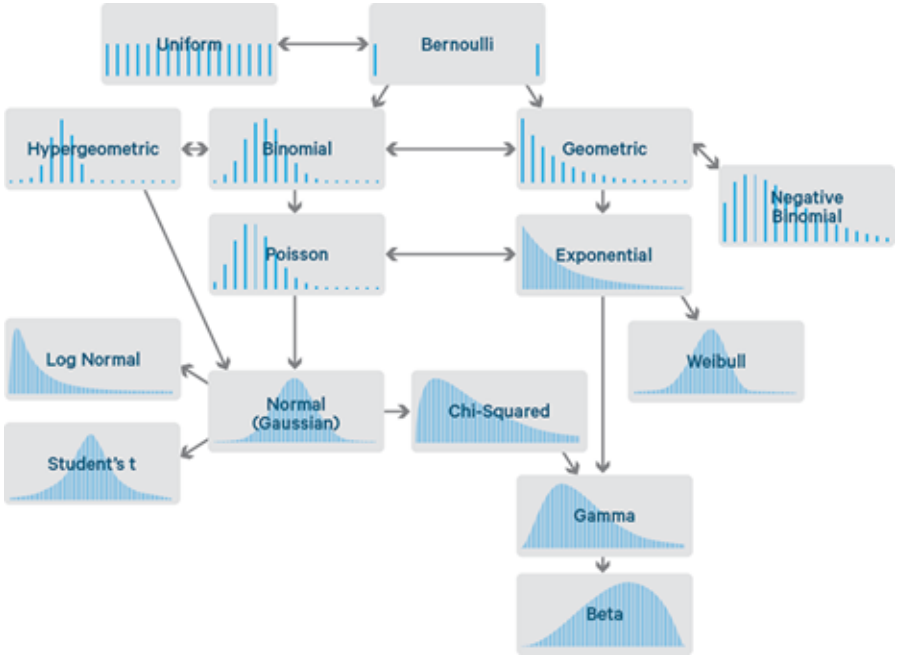
**Hanoi, 02/2024**

# Outline

- Supervised Learning
- Linear Regression with One Variable
  - Model Representation
  - Cost Functions
  - Gradient Descent
- Linear Regression with Multiple Variables
  - Learning rate
  - Normal Equation

# Recap: Random Variables



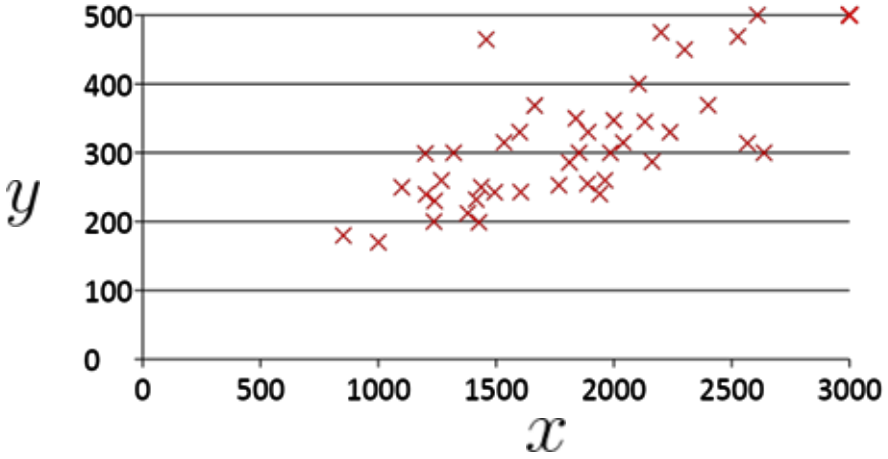
# Supervised Learning

- Supervised (Inductive) Learning
- Formalization
  - Input: $\mathbf{x} \in \mathcal{X} \quad \mathbb{R}^n$
  - Output: $y \in \mathcal{Y} \begin{cases} \mathbb{R} & \text{regression} \\ \{+1, -1\} & \text{binary classification} \\ \{1, 2, \ldots, K\} & \text{multi-class classification} \end{cases}$
  - Target function: $f : \mathcal{X} \to \mathcal{Y}$ (unknown)
  - Training Data: $D = \{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \ldots, (\mathbf{x}_m, y_m)\}$
  - Hypothesis: $h : \mathcal{X} \to \mathcal{Y} \qquad h \approx f$
  - Hypothesis space: $h \in \mathcal{H}$

# A Learning Problem




$h(x) \approx f(x)$



# The Statistical Learning Framework

A simple **data-generation pipeline**:

- Assume we have a probability distribution $\mathcal{D}$ over $\mathcal{X}$,
- Given $\mathcal{D}$, we draw a sample $x_i \sim \mathcal{D}$
- Assume we have a **correct** label function $c$:

$$c : \mathcal{X} \to \mathcal{Y} \tag{2}$$

that $y_i = c(x_i)$, and $S = ((x_1, y_1), \ldots, (x_m, y_m))$

Notes:

1. $c$ is function we want to recover. However, we do not know $\mathcal{D}$ and $c$, our learning algorithm $A$ can only access $\mathcal{X}$, $\mathcal{Y}$, $S$
2. Taken into account that, every time the **data-generation pipeline** runs, we can get a new training data $S$

# The Statistical Learning Framework

Given domain set $\mathcal{X}$ and label set $\mathcal{Y}$, the data distribution $\mathcal{D}$ and the labeling function $c$.

The error of a prediction rule $h : \mathcal{X} \to \mathcal{Y}$ is defined as:

$$L_{\mathcal{D},c}(h) \stackrel{\text{def}}{=} \mathbb{P}_{x \sim \mathcal{D}}[h(x) \neq c(x)] \tag{3}$$

However, we do not know $\mathcal{D}$ and $c$ in practice

# The Statistical Learning Framework

The Empirical Risk Minimization (ERM) framework learns a learner

$$h_S : \mathcal{X} \to \mathcal{Y}$$

that minimize the **the training error** without knowing $\mathcal{D}$ and $c$.

The **training error** is defined as

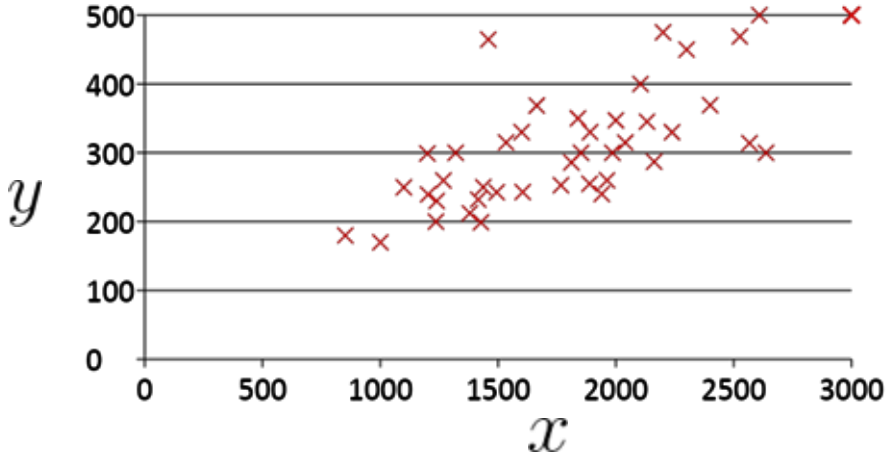
$$L_S(h) \stackrel{\text{def}}{=} \frac{|\{i \in [m] : h(x_i) \neq y_i\}|}{m} \tag{4}$$

Note that: we use $h_S$ to emphasize our learners depends on $S$, a specific $S$

# A Learning Problem




$h(x) \approx f(x)$



# Hypothesis Spaces

- Linear models

$$h(x) = ax + b \approx f(x)$$

  - **Infinite** possible hypotheses!
  - Any choices of coefficient a and b will result in a possible hypothesis
- Polynomial models

$$h(x) = ax^2 + bx + c \approx f(x)$$

- Any nonlinear models

$$h(x) = g(x) \approx f(x)$$

# Two Strategies for Machine Learning

- **Develop Languages for Expressing Prior Knowledge**
  - Rule grammars and stochastic models
- **Develop Flexible Hypothesis Spaces**
  - Nested collections of hypotheses, rules, linear models, decision trees, neural networks, etc.
- **For either case, the key is to**
  - Developing efficient algorithms for finding a Hypothesis that best approximates the target function for fitting the data

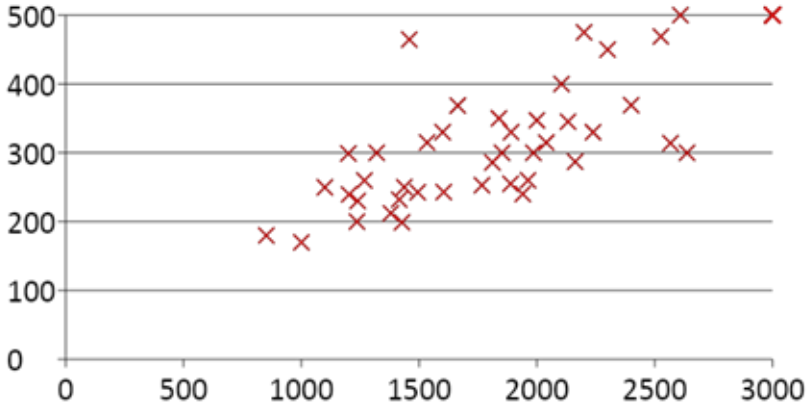
# Key Issues in Machine Learning

- What are good hypothesis spaces?
  - Which spaces have been useful in practical applications and why?
- What algorithms can work with these spaces?
  - Are there general design principles for machine learning algorithms?
- How can we find the best hypothesis in an efficient way?
  - How to find the optimal solution efficiently (“optimization” question)
- How can we optimize accuracy on future data?
  - Known as the “overfitting” problem (i.e., “generalization” theory)
- How can we have confidence in the results?
  - How much training data is required to find accurate hypothesis? (“statistical” question)
- Are some learning problems computationally intractable? (“computational” question)
- How can we formulate application problems as machine learning problems? (“engineering” question)

# Regression with One Variable (1)

**Housing Prices (Portland, OR)**

Price (in 1000s of dollars)




Size (feet$^2$)

**Supervised Learning**

Given the "right answer" for each example in the data.

**Regression Problem**

Predict real-valued output

# Regression with One Variable (2)

**Training set of housing prices (Portland, OR)**

| Size in feet$^2$ (x) | Price ($) in 1000's (y) |
|---|---|
| 2104 | 460 |
| 1416 | 232 |
| 1534 | 315 |
| 852 | 178 |
| … | … |

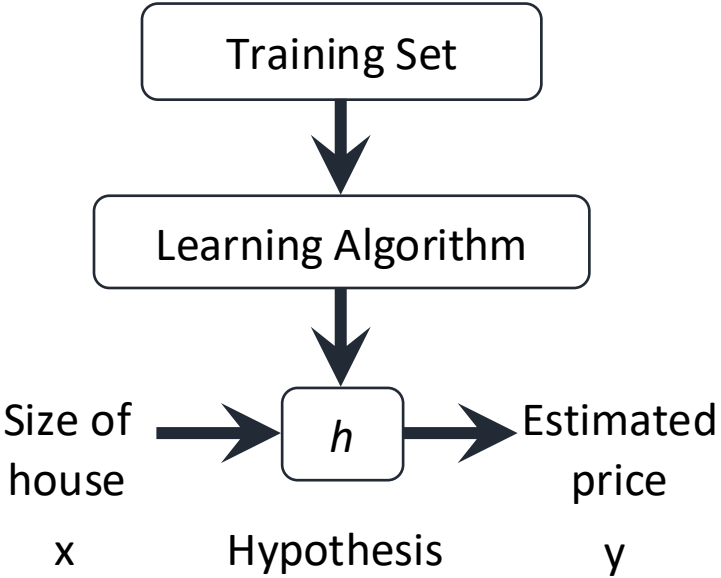
Notation:

**m** = Number of training examples

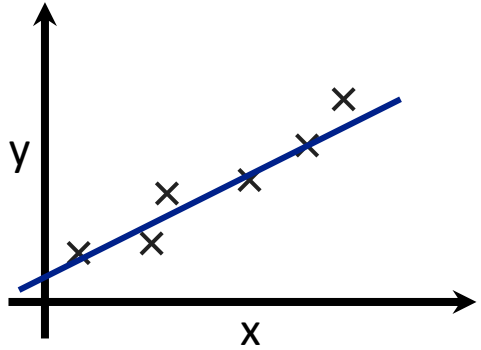
**x**'s = "input" variable / features

**y**'s = "output" variable / "target" variable

# Model Representation




**How do we represent *h* ?**




$$h(x) = w_0 + w_1 x$$

Linear regression with one variable.
"**Univariate Linear Regression**"

How to choose parameters $w_0, w_1$ ?

# Formulation: Cost Function (1)

Hypothesis:

$$h(x) = w_0 + w_1 x$$

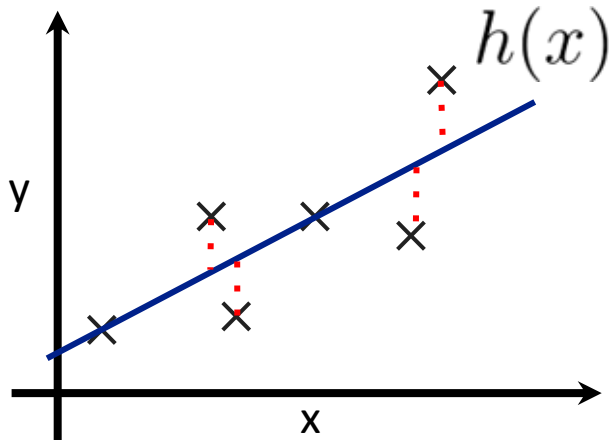
Parameters:

$$w_0, w_1$$

Cost Function: mean squared error (MSE)

$$J(w_0, w_1) = \frac{1}{2m} \sum_{i=1}^{m} (h(x_i) - y_i)^2$$

Goal: $\min_{w_0, w_1} J(w_0, w_1)$



# Formulation: Cost Function (2)

Hypothesis:

$$h(x) = w_0 + w_1 x$$

Parameters:

$$w_0, w_1$$

Cost Function:

$$J(w_0, w_1) = \frac{1}{2m} \sum_{i=1}^{m} (h(x_i) - y_i)^2$$

Goal:

$$\min_{w_0, w_1} J(w_0, w_1)$$

Simplified

$$h(x) = w_1 x$$

$$w_1$$

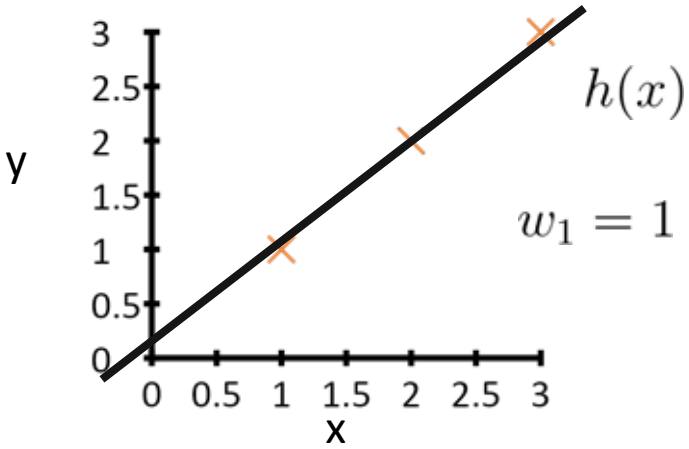
$$J(w_1) = \frac{1}{2m} \sum_{i=1}^{m} (h(x_i) - y_i)^2$$

$$\min_{w_1} J(w_1)$$

# Cost Function: Example (1)

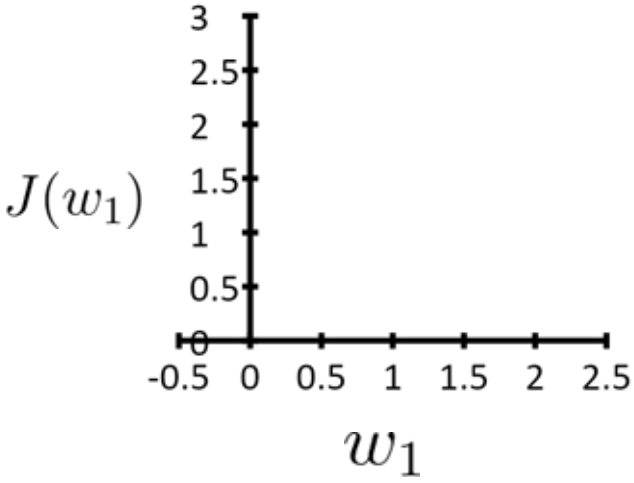
$h(x)$

For fix $w_1$ this is a function of x




$J(w_1)$

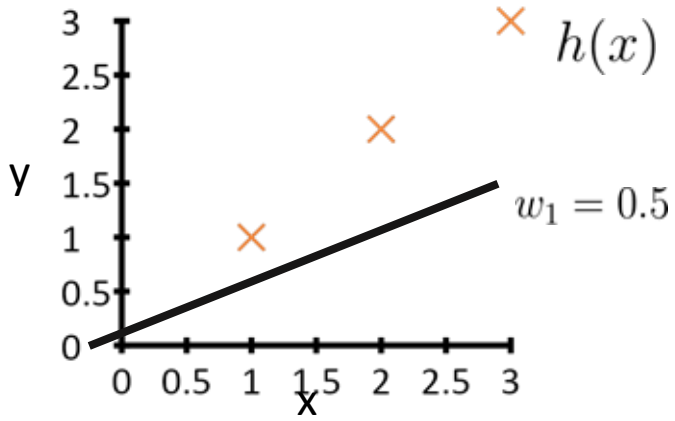
function of the parameter $w_1$



# Cost Function: Example (2)

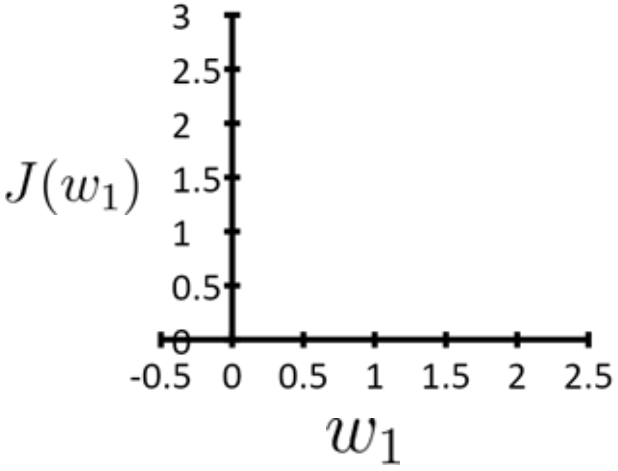
$h(x)$

For fix $w_1$ this is a function of x




$J(w_1)$

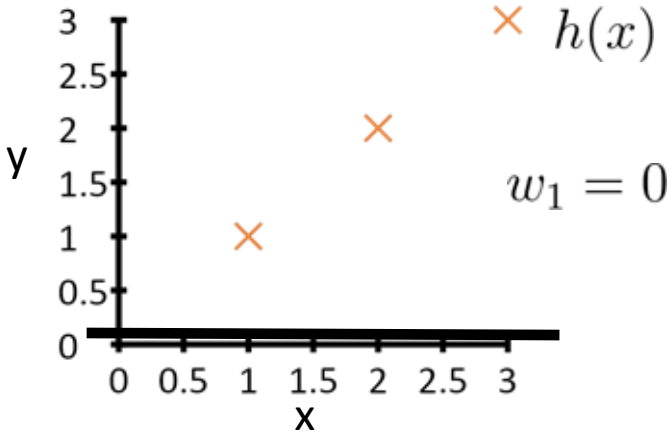
function of the parameter $w_1$



# Cost Function: Example (3)

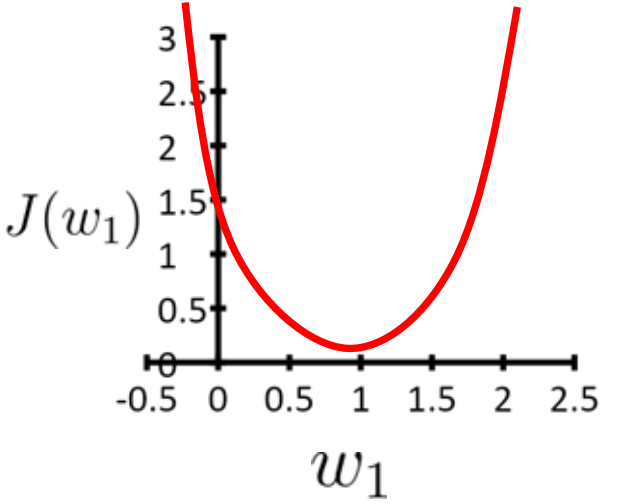
$h(x)$

For fix $w_1$ this is a function of x




$J(w_1)$

function of the parameter $w_1$



# Cost Function (1)

Hypothesis: $h(x) = w_0 + w_1 x$

Parameters: $w_0, w_1$

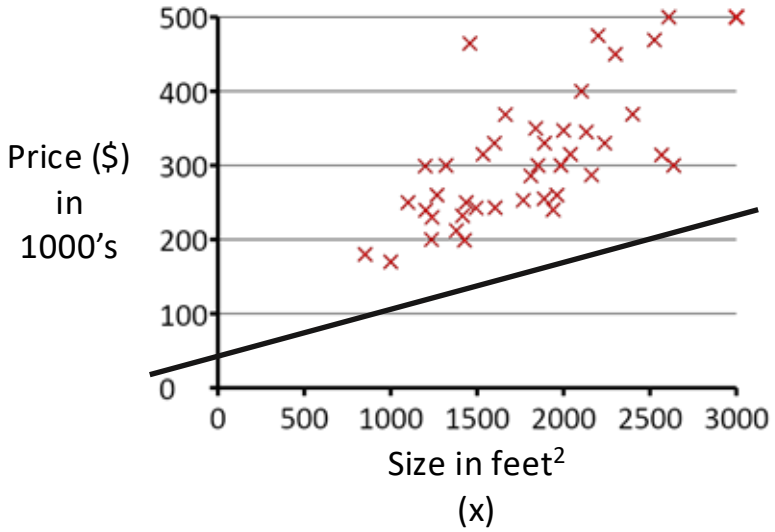
Cost Function: $J(w_0, w_1) = \frac{1}{2m} \sum_{i=1}^{m} \left(h(x_i) - y_i\right)^2$

Goal: $\min_{w_0, w_1} J(w_0, w_1)$

# Cost Function (2)

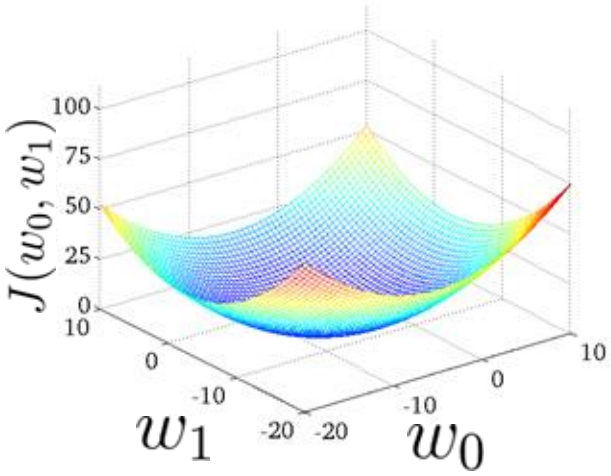
$h(x)$

(for fixed $w_0, w_1$ this is a function of x)




$J(w_0, w_1)$

(function of the parameters $w_0, w_1$ )

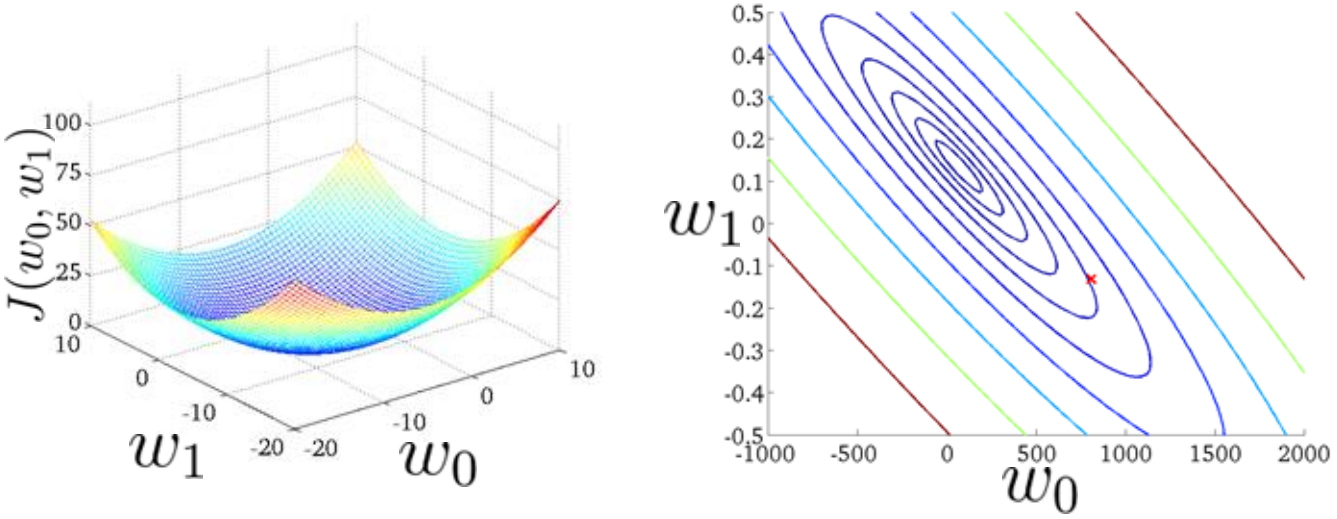



$$h(x) = 50 + 0.06x$$

# Cost Function (3)

- Contour plots

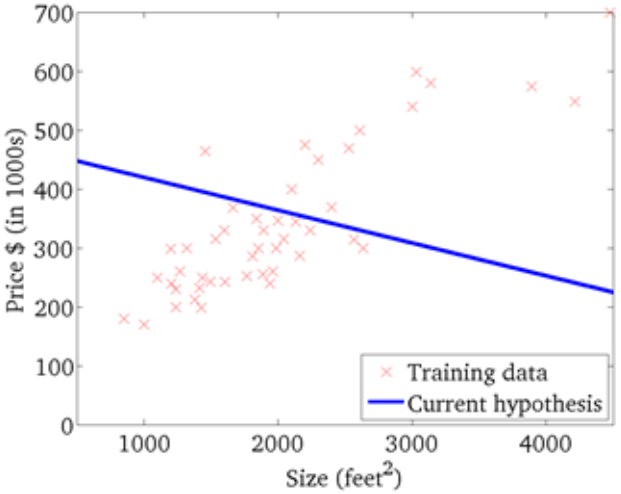
$$J(w_0, w_1)$$



# Cost Function (4)

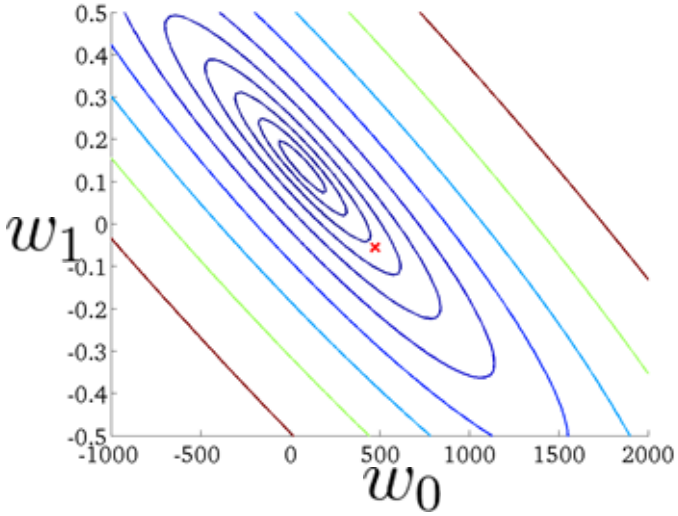
$h(x)$

(for fixed $w_0, w_1$ , this is a function of x)




$J(w_0, w_1)$

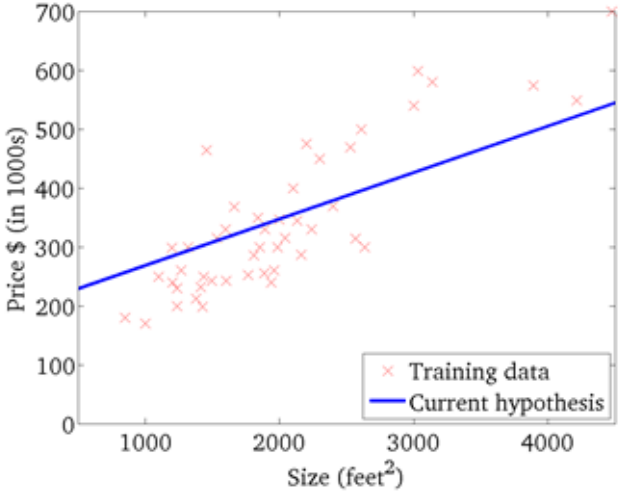
(function of the parameters $w_0, w_1$ )



# Cost Function (5)

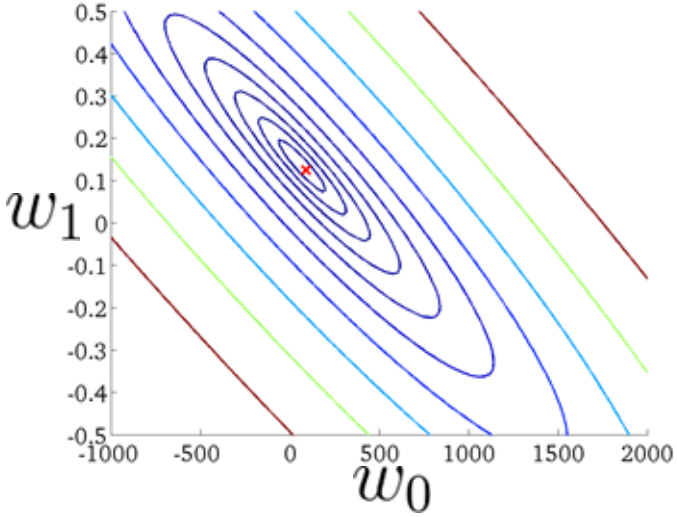
$h(x)$

(for fixed $w_0, w_1$, this is a function of x)




$J(w_0, w_1)$

(function of the parameters $w_0, w_1$ )



# Gradient Descent for Optimization (1)

Given some objective function $J(w_0, w_1)$

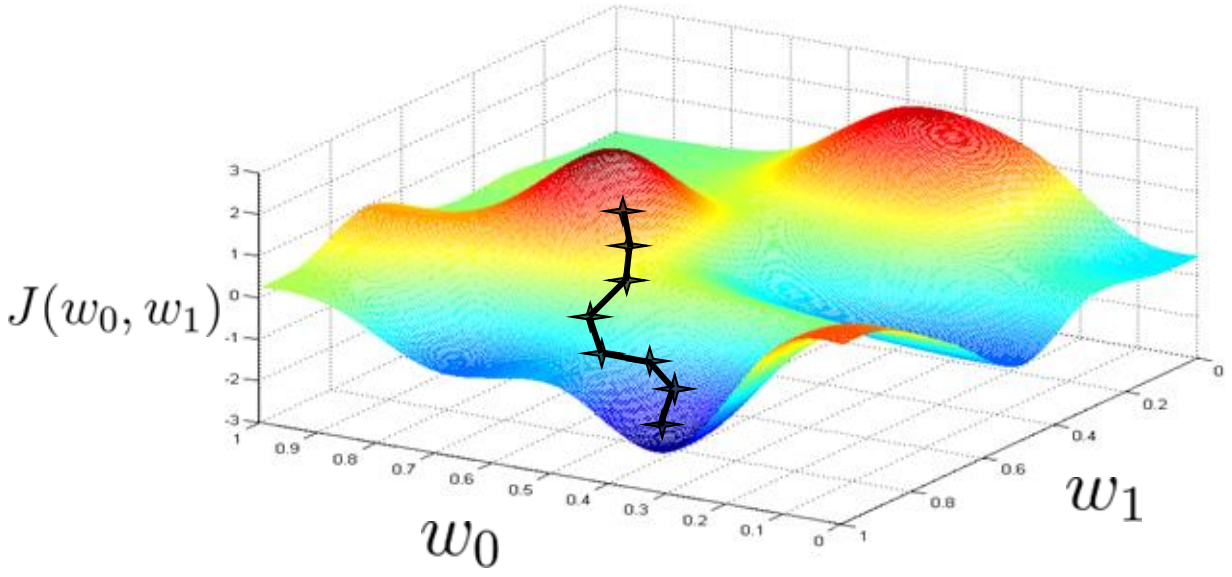
Want to optimize $\min_{w_0, w_1} J(w_0, w_1)$

**Outline:**

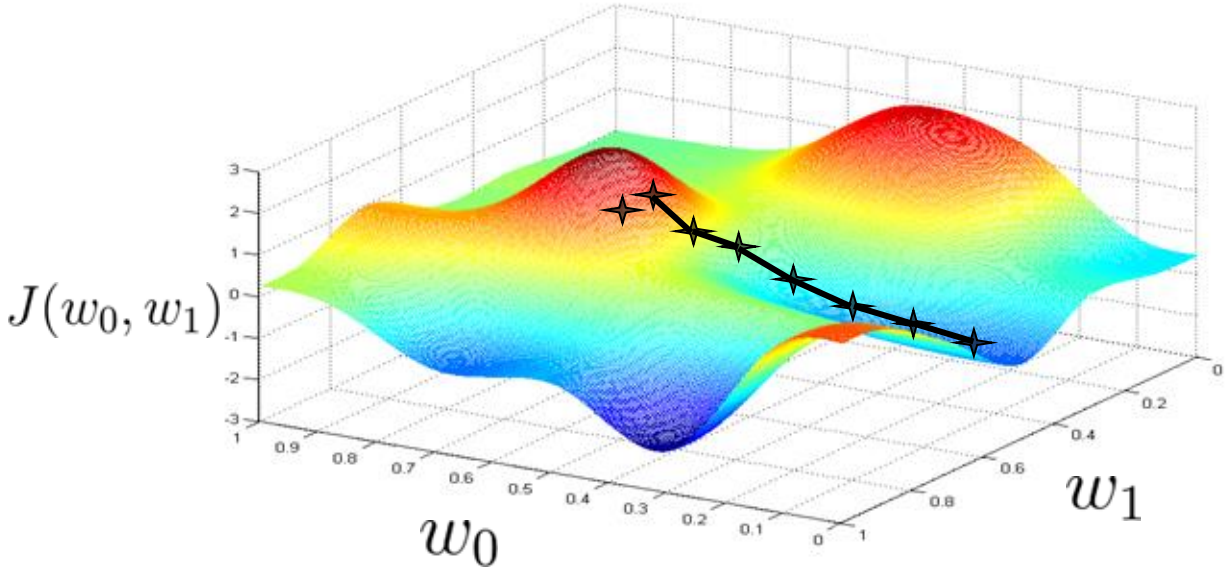
- Start with some $w_0, w_1$
- Keep changing $w_0, w_1$ to reduce $J(w_0, w_1)$

  until we hopefully end up at a minimum

# Gradient Descent for Optimization (2)



# Gradient Descent for Optimization (3)



# Gradient Descent Algorithm

## Gradient descent algorithm

initialize $w_j \quad j = 0, 1$

repeat until convergence {

$$w_j := w_j - \alpha \frac{\partial}{\partial w_j} J(w_0, w_1) \quad \text{(simultaneously update } j = 0 \text{ and } j = 1\text{)}$$

}

learning rate parameter
(rule of thumb: 0.1)

# Gradient Descent for Linear Regression (1)

Gradient descent algorithm

repeat until convergence {

$$w_j := w_j - \alpha \frac{\partial}{\partial w_j} J(w_0, w_1)$$

(for $j = 1$ and $j = 0$)

}

Linear Regression Model

$$h(x) = w_0 + w_1 x$$

$$J(w_0, w_1) = \frac{1}{2m} \sum_{i=1}^{m} \left(h(x_i) - y_i\right)^2$$

$$\frac{\partial}{\partial w_j} J(w_0, w_1) = \frac{\partial}{\partial w_j} \left( \frac{1}{2m} \sum_{i=1}^{m} \left((w_0 + w_1 x_i) - y_i\right)^2 \right)$$

$$\frac{\partial}{\partial w_0} J(w_0, w_1) = \frac{1}{m} \sum_{i=1}^{m} (h(x_i) - y_i)$$

$$\frac{\partial}{\partial w_1} J(w_0, w_1) = \frac{1}{m} \sum_{i=1}^{m} (h(x_i) - y_i) \cdot x_i$$

# Gradient Descent for Linear Regression (2)

**Gradient descent algorithm**

$$\frac{\partial}{\partial w_0} J(w_0, w_1)$$

repeat until convergence {

$$w_0 := w_0 - \alpha \frac{1}{m} \sum_{i=1}^{m} \left(h(x_i) - y_i\right)$$

$$w_1 := w_1 - \alpha \frac{1}{m} \sum_{i=1}^{m} \left(h(x_i) - y_i\right) \cdot x_i$$

}

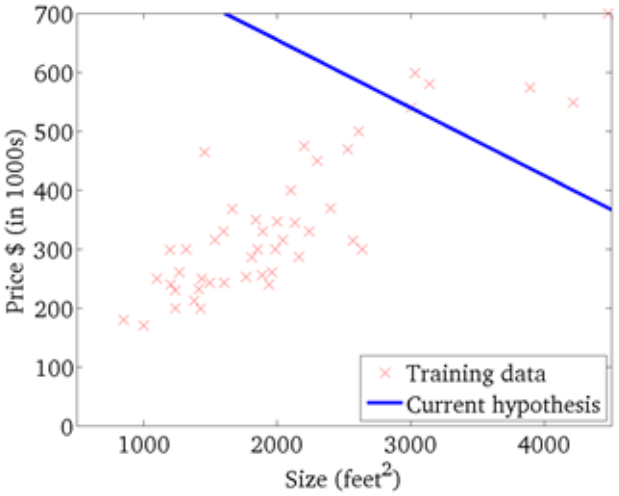
update $w_0$ and $w_1$ simultaneously

$$\frac{\partial}{\partial w_1} J(w_0, w_1)$$

# Gradient Descent Example (1)

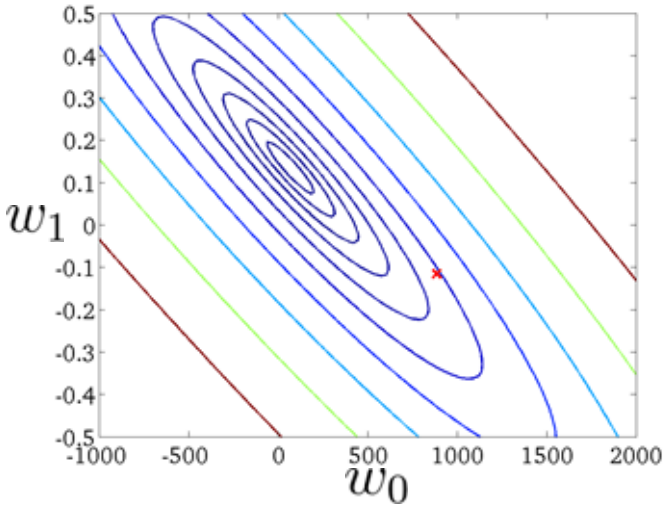
$h(x)$

(for fixed $w_0, w_1$, this is a function of x)




$J(w_0, w_1)$

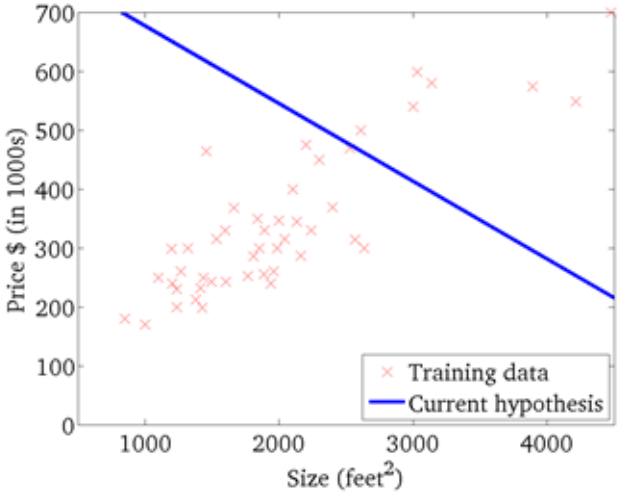
(function of the parameters $w_0, w_1$ )



# Gradient Descent Example (2)

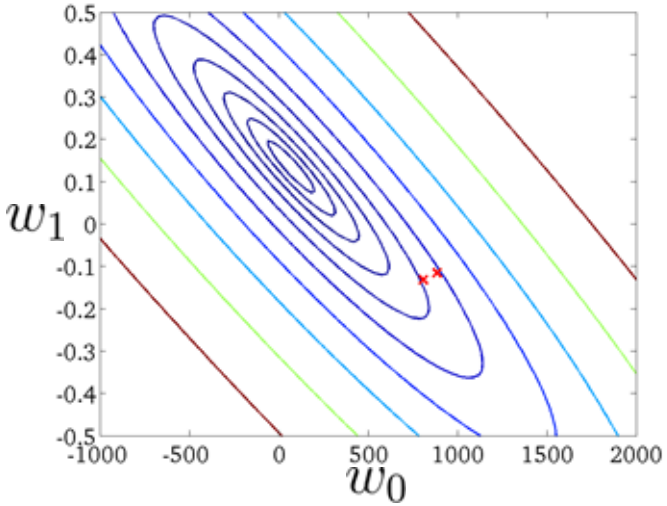
$h(x)$

(for fixed $w_0, w_1$, this is a function of x)




$J(w_0, w_1)$

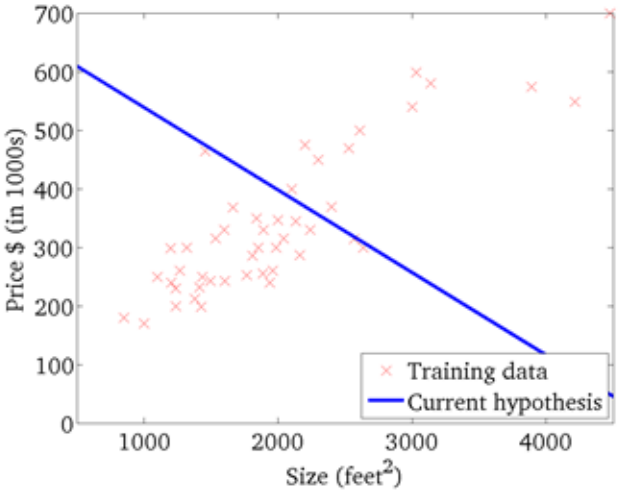
(function of the parameters $w_0, w_1$ )



# Gradient Descent Example (3)

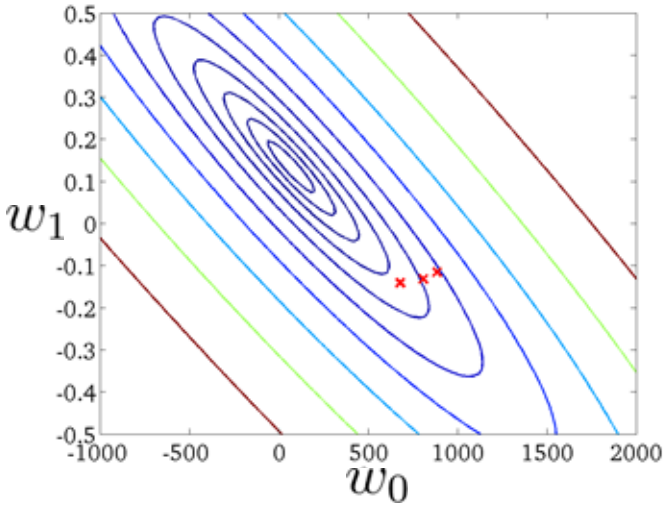
$h(x)$

(for fixed $w_0, w_1$, this is a function of x)




$J(w_0, w_1)$

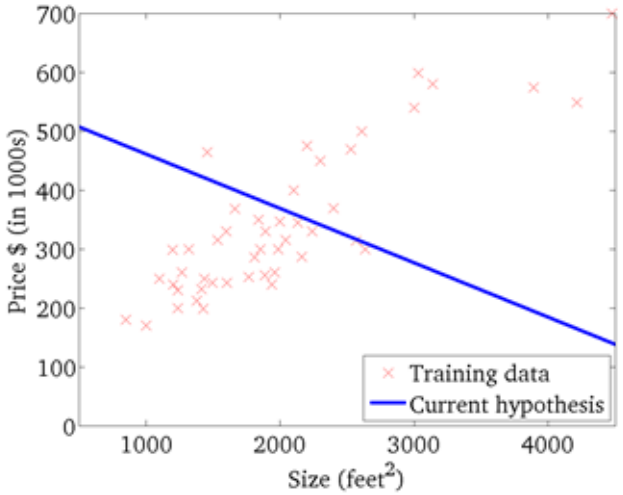
(function of the parameters $w_0, w_1$ )



# Gradient Descent Example (4)

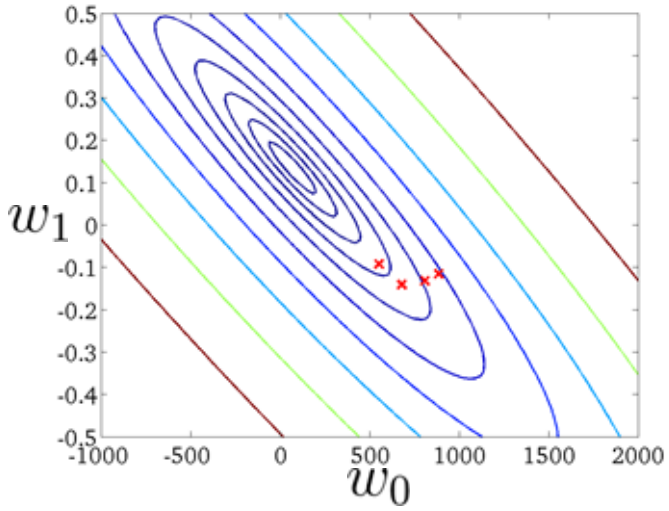
$h(x)$

(for fixed $w_0, w_1$ , this is a function of x)




$J(w_0, w_1)$

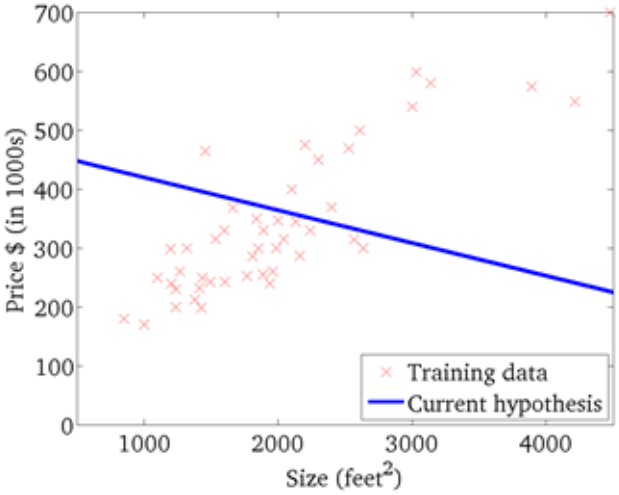
(function of the parameters $w_0, w_1$ )



# Gradient Descent Example (5)

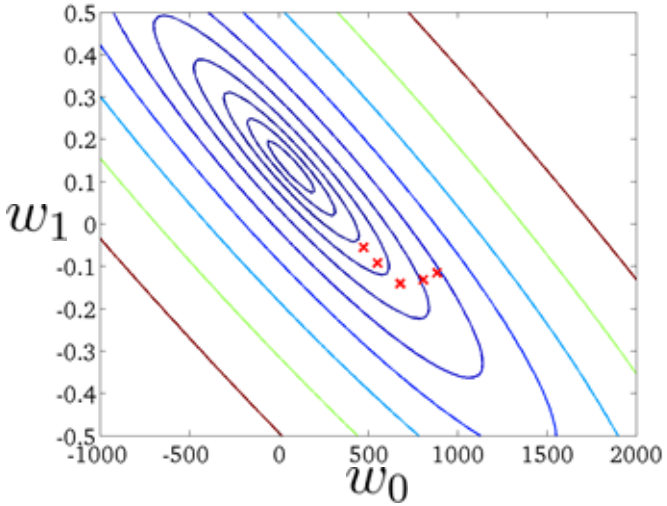
$h(x)$

(for fixed $w_0, w_1$ , this is a function of x)




$J(w_0, w_1)$

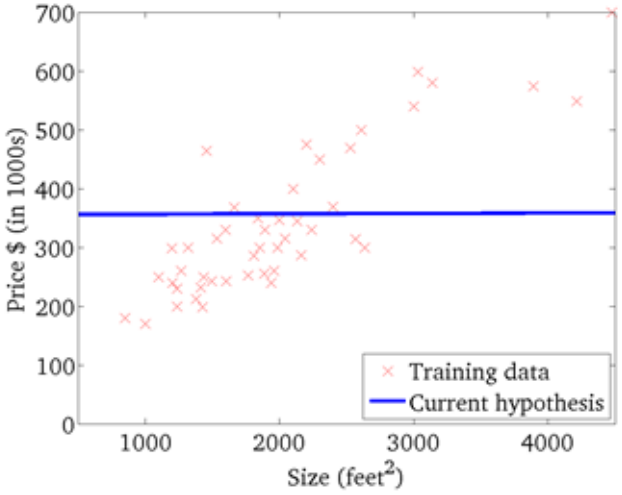
(function of the parameters $w_0, w_1$ )



# Gradient Descent Example (6)

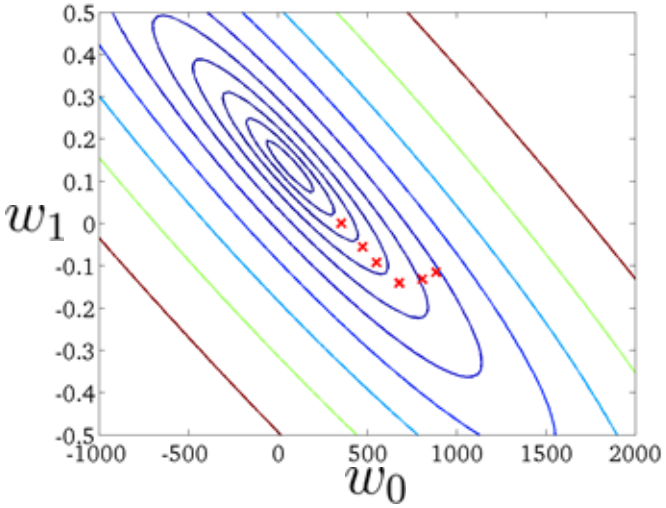
$h(x)$

(for fixed $w_0, w_1$, this is a function of x)




$J(w_0, w_1)$

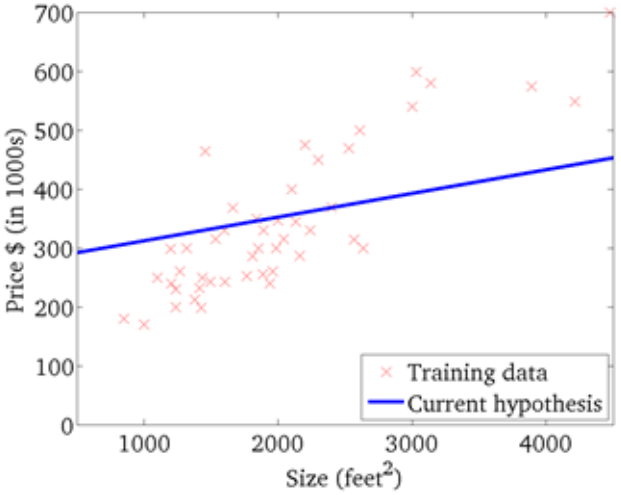
(function of the parameters $w_0, w_1$ )



# Gradient Descent Example (7)

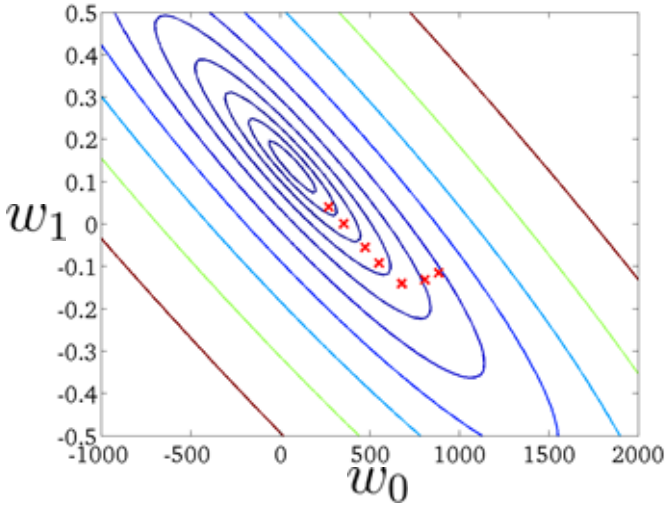
$h(x)$

(for fixed $w_0, w_1$, this is a function of x)




$J(w_0, w_1)$

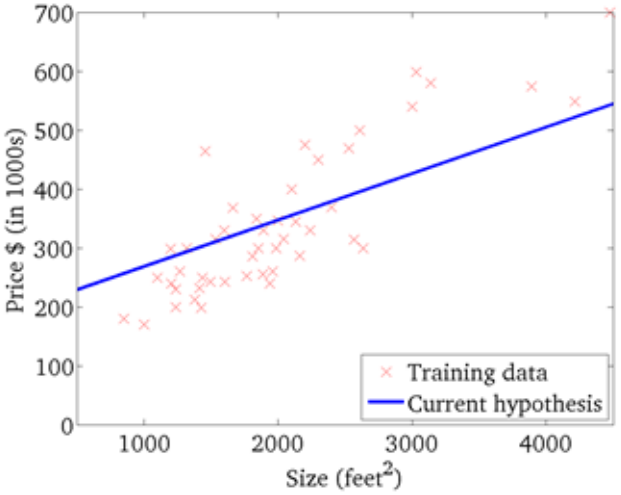
(function of the parameters $w_0, w_1$ )



# Gradient Descent Example (8)

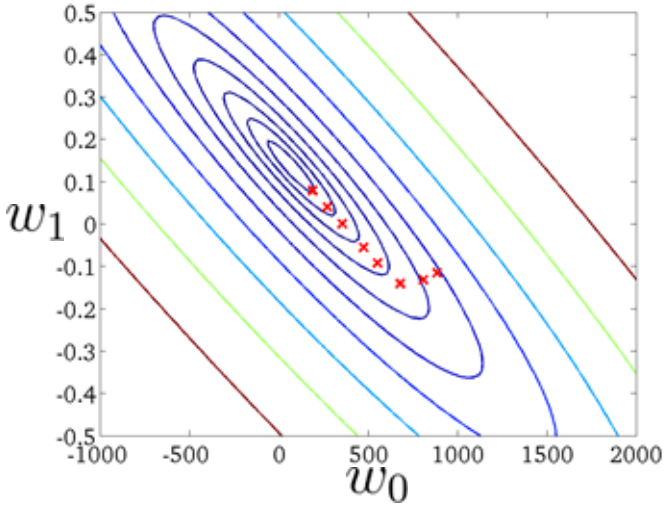
$h(x)$

(for fixed $w_0, w_1$, this is a function of x)




$J(w_0, w_1)$

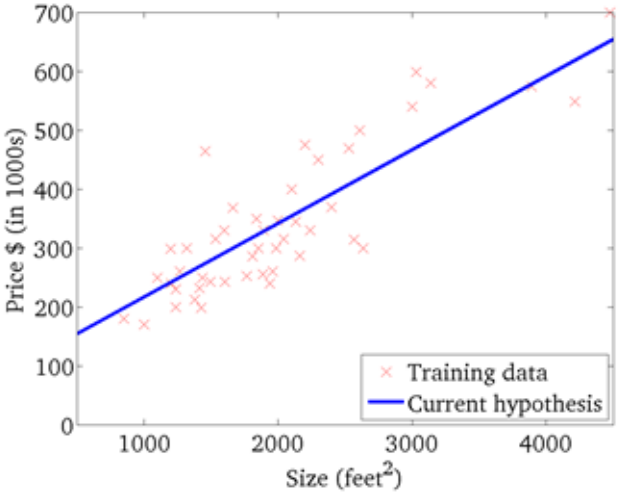
(function of the parameters $w_0, w_1$ )



# Gradient Descent Example (9)

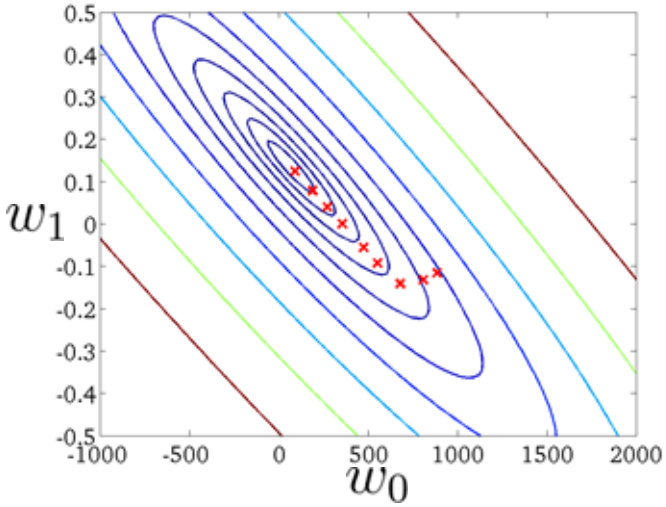
$h(x)$

(for fixed $w_0, w_1$, this is a function of x)




$J(w_0, w_1)$

(function of the parameters $w_0, w_1$ )



# Batch Gradient Descent

"Batch": Each step of gradient descent uses all the training examples.

$$
\begin{aligned}
&\text{repeat until convergence } \{ \\
&\quad w_0 := w_0 - \alpha \frac{1}{m} \sum_{i=1}^{m} \left( h(x_i) - y_i \right) \\
&\quad w_1 := w_1 - \alpha \frac{1}{m} \sum_{i=1}^{m} \left( h(x_i) - y_i \right) \cdot x_i \\
&\}
\end{aligned}
$$

# Multivariate Linear Regression (1)

**Multiple features (variables).**

| Size (feet$^2$) | Number of bedrooms | Number of floors | Age of home (years) | Price ($1000) $y$ |
|---|---|---|---|---|
| 2104 | 5 | 1 | 45 | 460 |
| 1416 | 3 | 2 | 40 | 232 |
| 1534 | 3 | 2 | 30 | 315 |
| 852 | 2 | 1 | 36 | 178 |
| … | … | … | … | … |

Notation:

$n$ = number of features

$\mathbf{x}_i$ = input (features) of $i^{th}$ training example.

$x_{ij}$ = value of feature $j$ in $i^{th}$ training example.

# Multivariate Linear Regression (2)

Hypothesis:

Previously: $h(x) = w_0 + w_1 x$

$\mathbf{x} \in \mathbb{R}^n$ $\quad h(\mathbf{x}) = w_0 + w_1 x_1 + w_2 x_2 + \ldots + w_n x_n$

For convenience of notation, define $x_0 = 1$ .

$$h(\mathbf{x}) = \sum_{j=0}^{n} w_j x_j = \mathbf{w}^T \mathbf{x} = \langle \mathbf{w}, \mathbf{x} \rangle$$

$$\mathbf{x} \in \mathbb{R}^{n+1} \quad \mathbf{w} \in \mathbb{R}^{n+1}$$

# Gradient Descent for Multivariate LR

Hypothesis: $h(\mathbf{x}) = \mathbf{w}^T\mathbf{x}$

Parameters: $\mathbf{w} = (w_0, w_1, \ldots, w_n)^T$

Cost function: $J(\mathbf{w}) = \frac{1}{2m}\sum^{m}(h(\mathbf{x}_i) - y_i)^2$

Gradient descent: $w_j := w_j - \alpha\frac{\partial}{\partial w_j}J(\mathbf{w})$

Repeat $\{$ (simultaneously update for every $j = 0, \ldots, n$)

$$\mathbf{w} := \mathbf{w} - \alpha\nabla J(\mathbf{w})$$

$\}$

$$\nabla J(\mathbf{w}) = \left(\frac{\partial}{\partial w_0}J(\mathbf{w}), \frac{\partial}{\partial w_1}J(\mathbf{w}), \ldots, \frac{\partial}{\partial w_n}J(\mathbf{w})\right)$$

# Univariate LR vs Multivariate LR

## Gradient Descent

Previously (n=1):

Repeat $\{$

$$w_0 := w_0 - \alpha \underbrace{\frac{1}{m}\sum_{i=1}^{m}\left(h(x_i) - y_i\right)}_{\frac{\partial}{\partial w_0}J(w_0, w_1)}$$

$$w_1 := w_1 - \alpha \underbrace{\frac{1}{m}\sum_{i=1}^{m}\left(h(x_i) - y_i\right)\cdot x_i}_{\frac{\partial}{\partial w_1}J(w_0, w_1)}$$

$\}$

(simultaneously update $\theta_0, \theta_1$)

New algorithm $(n \geq 1)$:

Repeat $\{$

$$\mathbf{w} := \mathbf{w} - \alpha \nabla J(\mathbf{w})$$

$\}$

$$\nabla J(\mathbf{w}) = \left(\frac{\partial}{\partial w_0}J(\mathbf{w}), \frac{\partial}{\partial w_1}J(\mathbf{w}), \ldots, \frac{\partial}{\partial w_n}J(\mathbf{w})\right)$$

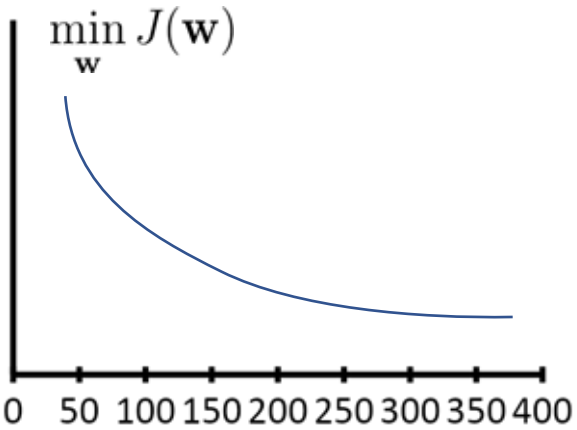
$$\frac{\partial}{\partial w_0}J(\mathbf{w}) = \frac{1}{m}\sum_{i=1}^{m}(h(\mathbf{x}_i) - y_i)$$

$$\frac{\partial}{\partial w_1}J(\mathbf{w}) = \frac{1}{m}\sum_{i=1}^{m}(h(\mathbf{x}_i) - y_i)x_{i1}$$

$$\vdots$$

# Convergence and Learning Rate

$$\mathbf{w} := \mathbf{w} - \alpha \nabla J(\mathbf{w})$$




Example automatic convergence test:

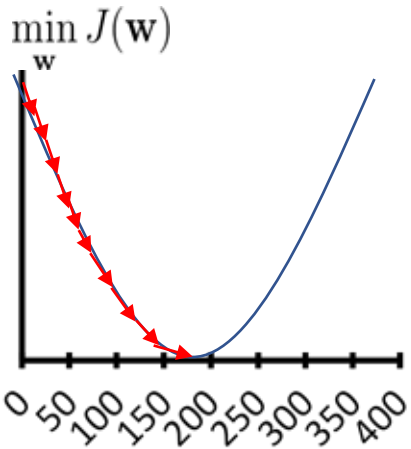
Declare convergence if $J(\mathbf{w})$ decreases by less than $10^{-3}$ in one iteration.

For sufficiently small $\alpha$, $J(\mathbf{w})$ should decrease on every iteration.

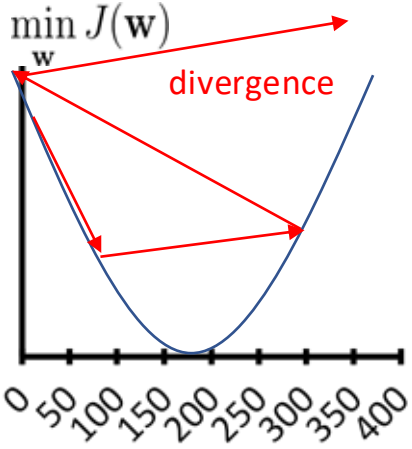
But if $\alpha$ is too small, gradient descent can be slow to converge.

If $\alpha$ is too large: $J(\mathbf{w})$ may not decrease on every iteration; may not converge.

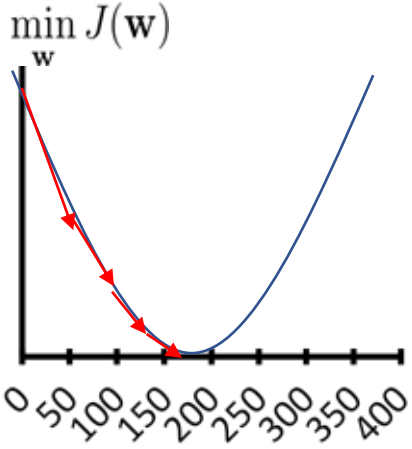
# Learning Rate




too small
constant




too
large




gradually
decreased $\alpha_t = \frac{\alpha}{t}$

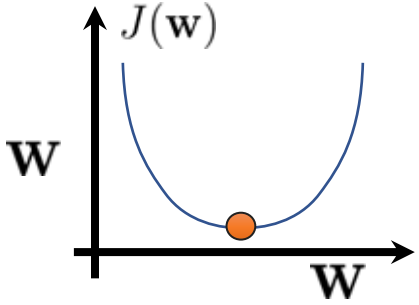
# Normal Equation (1)

**Gradient Descent**

- Iterative approach

**Normal Equation**

- Analytical method to solve




Intuition Example: If 1D

$$J(w) = aw^2 + bw + c$$

---

$$\mathbf{w} \in \mathbb{R}^{n+1} \quad J(\mathbf{w}) = \frac{1}{2m}\sum_{i=1}^{m}(h(\mathbf{x}_i) - y_i)^2 = \frac{1}{2m}\sum_{i=1}^{m}(\mathbf{w}^T\mathbf{x}_i - y_i)^2$$

$$\nabla J(\mathbf{w}) = 0 \longrightarrow$$ **Solve equation to find w**

# Normal Equation (2)

$$J(\mathbf{w}) = \frac{1}{2m}\sum_{i=1}^{m}(\mathbf{w}^T\mathbf{x}_i - y_i)^2 = \frac{1}{2m}(X\mathbf{w} - \mathbf{y})^T(X\mathbf{w} - \mathbf{y})$$

$$(\mathbf{w}^T\mathbf{x}_1 - y_1)$$

$$X\mathbf{w} - \mathbf{y} = \begin{pmatrix} x_{10} & x_{11} & \dots & x_{1n} \\ x_{20} & x_{21} & \dots & x_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ x_{m0} & x_{n1} & \dots & x_{mn} \end{pmatrix} \begin{pmatrix} w_0 \\ w_1 \\ \vdots \\ w_n \end{pmatrix} - \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_m \end{pmatrix}$$

$m \times (n+1)$ $\quad$ $(n+1) \times 1$ $\quad$ $m \times 1$

# Normal Equation (3)

- Matrix-vector formulation

$$J(\mathbf{w}) = \frac{1}{2m}(X\mathbf{w} - \mathbf{y})^T(X\mathbf{w} - \mathbf{y})$$

$$\nabla J(\mathbf{w}) = \nabla_{\mathbf{w}}\left(\frac{1}{2m}(X\mathbf{w} - \mathbf{y})^T(X\mathbf{w} - \mathbf{y})\right)$$

$$= X^T X\mathbf{w} - X^T\mathbf{y} = \mathbf{0}$$

$$X^T X\mathbf{w} = X^T\mathbf{y}$$

- Analytical solution

$$\mathbf{w} = \left((X^T X)^{-1}X^T\right)\mathbf{y} = X^{\dagger}\mathbf{y}$$

$$X^{\dagger} = (X^T X)^{-1}X^T$$

# The Pseudo-inverse

$$\mathbf{w} = \left((X^T X)^{-1} X^T\right) \mathbf{y} = X^\dagger \mathbf{y}$$

$$X^\dagger = (X^T X)^{-1} X^T$$

$$\underbrace{\Big(\underbrace{\big[X^T X\big]}_{(n+1)\times(n+1)}\Big)^{-1}}_{(n+1)\times m} \underbrace{\big[\quad X^T \quad\big]}_{(n+1)\times m}$$

# Normal Equation: Example

Examples: $m = 4$.

| | Size (feet$^2$) | Number of bedrooms | Number of floors | Age of home (years) | Price ($1000) |
|---|---|---|---|---|---|
| $x_0$ | $x_1$ | $x_2$ | $x_3$ | $x_4$ | $y$ |
| 1 | 2104 | 5 | 1 | 45 | 460 |
| 1 | 1416 | 3 | 2 | 40 | 232 |
| 1 | 1534 | 3 | 2 | 30 | 315 |
| 1 | 852 | 2 | 1 | 36 | 178 |

$$X = \begin{bmatrix} 1 & 2104 & 5 & 1 & 45 \\ 1 & 1416 & 3 & 2 & 40 \\ 1 & 1534 & 3 & 2 & 30 \\ 1 & 852 & 2 & 1 & 36 \end{bmatrix} \qquad y = \begin{bmatrix} 460 \\ 232 \\ 315 \\ 178 \end{bmatrix}$$

$\mathbf{w} = \left((X^T X)^{-1} X^T\right) \mathbf{y} = X^{\dagger} \mathbf{y}$ $(X^T X)^{-1}$ is inverse of matrix $X^T X$ .

# Gradient Descent vs Normal Equation

$m$ **training examples,** $n$ **features.**

| Gradient Descent | Normal Equation |
| --- | --- |
| • Need to choose $\alpha$.<br>• Needs many iterations.<br>• Works well even when $n$ is large. | • No need to choose $\alpha$.<br>• Don't need to iterate.<br>• Need to compute $(X^T X)^{-1}$<br>• Slow if $n$ is very large. |

# Summary

- Supervised Learning
- Linear Regression with One Variable
  - Model Representation
  - Cost Functions
  - Gradient Descent
- Linear Regression with Multiple Variables
  - Learning rate
  - Normal Equation